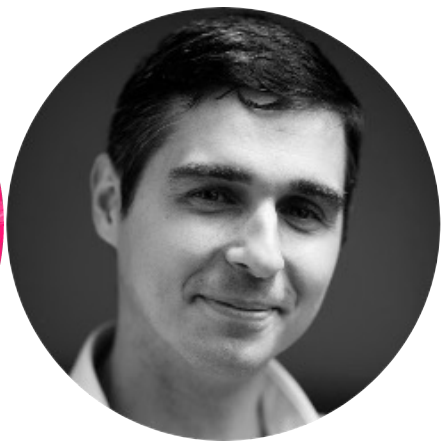


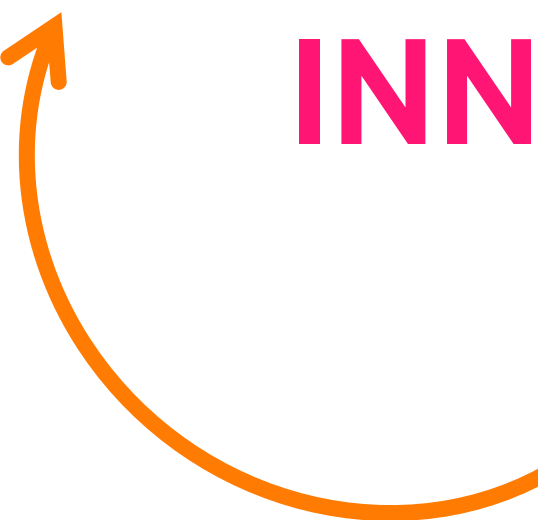
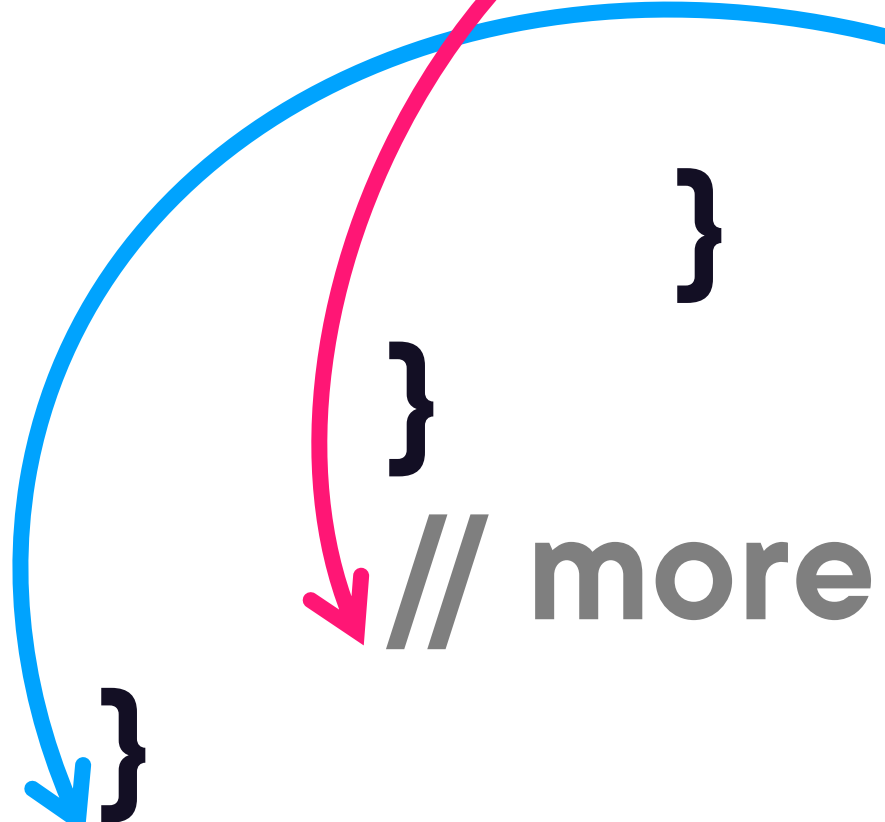
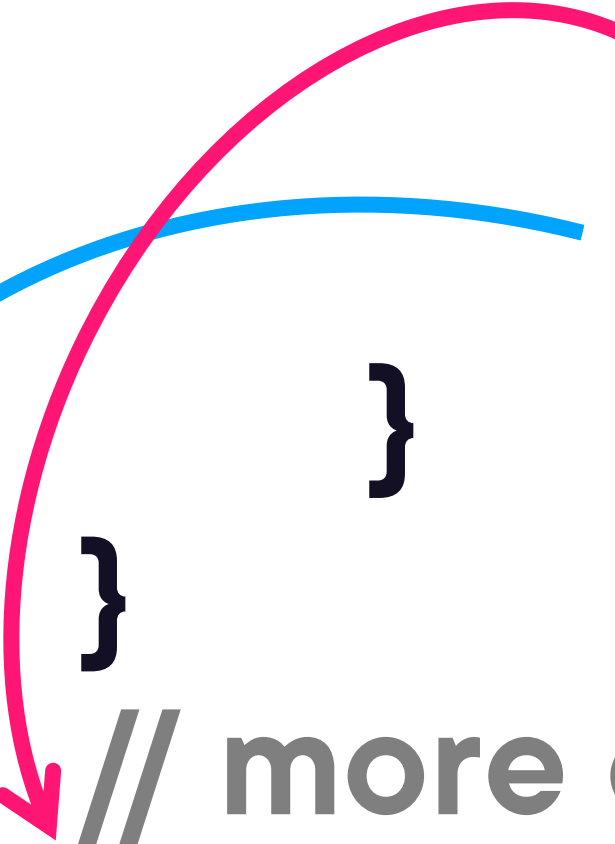
Understanding Advanced Flow Control



Andrejs Doronins

Software Developer in Test



```
void doThing() {  
    OUTER: for (init;evaluate;update) {  
        INNER: for(init;evaluate;update) {  
            continue;   
            break;   
            break OUTER ;  
            return;   
        }  
    }  
    // more code   
}
```

Questions with a nested loop? Skip and come back later!



Building Blocks

```
for (init; eval; update) {  
    for (init; eval; update) {  
        if (condition) {  
            // ...  
        }  
    }  
}
```



Nested Loops

```
for (init; eval; update) {  
    for (init; eval; update) {  
    }  
}
```



```
int[][] nestedArray = {  
    {1, 2},  
    {3, 4},  
    {5, 6}  
};
```

```
for (int i = 0; i < nestedArray.length; i++) {  
    for (int j = 0; j < nestedArray[i].length; j++) {  
        System.out.print(nestedArray[i][j] + " ");  
    }  
    System.out.println();  
}
```

Stay inside the
inner loop



```
int[][] nestedArray = {  
    {1, 2},  
    {3, 4},  
    {5, 6}  
};  
  
for(int[] innerArray : nestedArray) {  
    for (int j = 0; j < innerArray.length; j++) {  
        System.out.print(innerArray[j] + " ");  
    }  
    System.out.println();  
}
```



```
int[][] nestedArray = {  
    {1, 2},  
    {3, 4},  
    {5, 6}  
};
```

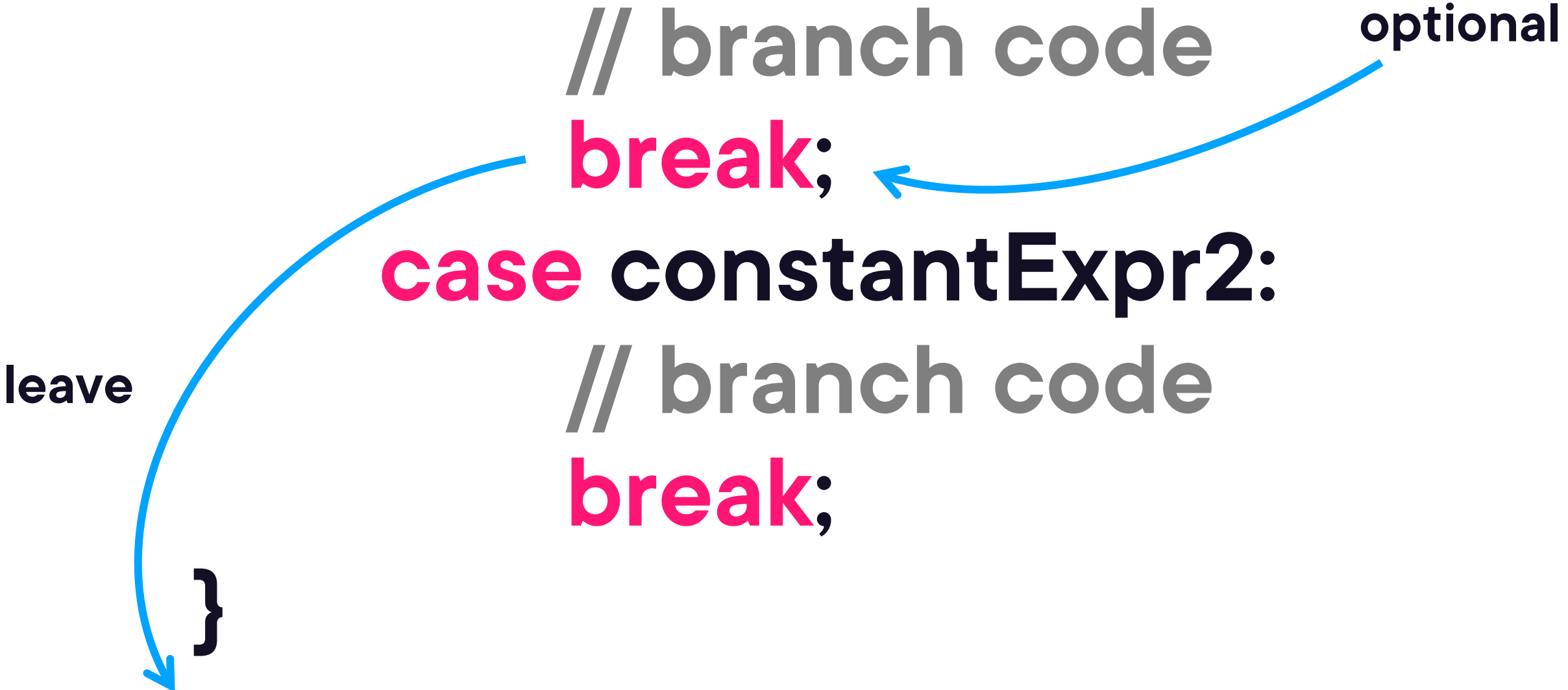
```
OUTER: for(int[] innerArray : nestedArray) {  
    INNER: for (int j = 0; j < innerArray.length; j++) {  
        System.out.print(innerArray[j] + " ");  
    }  
    System.out.println();  
}
```



```
switch (testVar) {  
    case constantExpr1:  
        // branch code  
        break;  
    case constantExpr2:  
        // branch code  
        break;  
}
```

optional

leave



break

Breaks out of the **enclosing** statement.



```
int[][] nestedArray = { {1, 2}, {3, 4}, {5, 6} };
```

```
OUTER: for (int i = 0; i < nestedArray.length; i++) {  
    INNER: for (int j = 0; j < nestedArray[i].length; j++) {  
        if (nestedArray[i][j] == 3){  
            break;  
        }  
        System.out.print(nestedArray[i][j] + " ");  
    }  
    System.out.println();  
}
```



Output

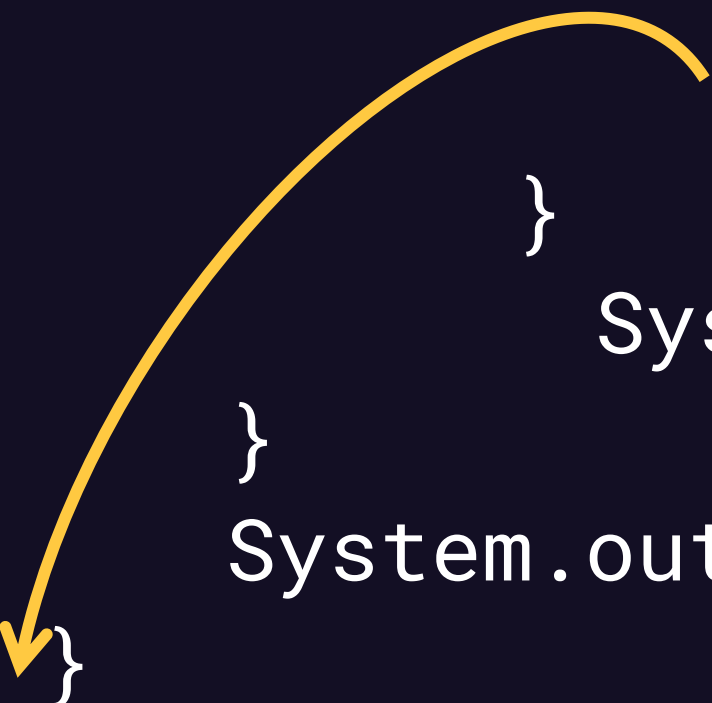
1 2

5 6



```
int[][] nestedArray = { {1, 2}, {3, 4}, {5, 6} };
```

```
OUTER: for (int i = 0; i < nestedArray.length; i++) {  
    INNER: for (int j = 0; j < nestedArray[i].length; j++) {  
        if (nestedArray[i][j] == 3){  
            break OUTER;  
        }  
        System.out.print(nestedArray[i][j] + " ");  
    }  
    System.out.println();  
}
```



Output

1 2



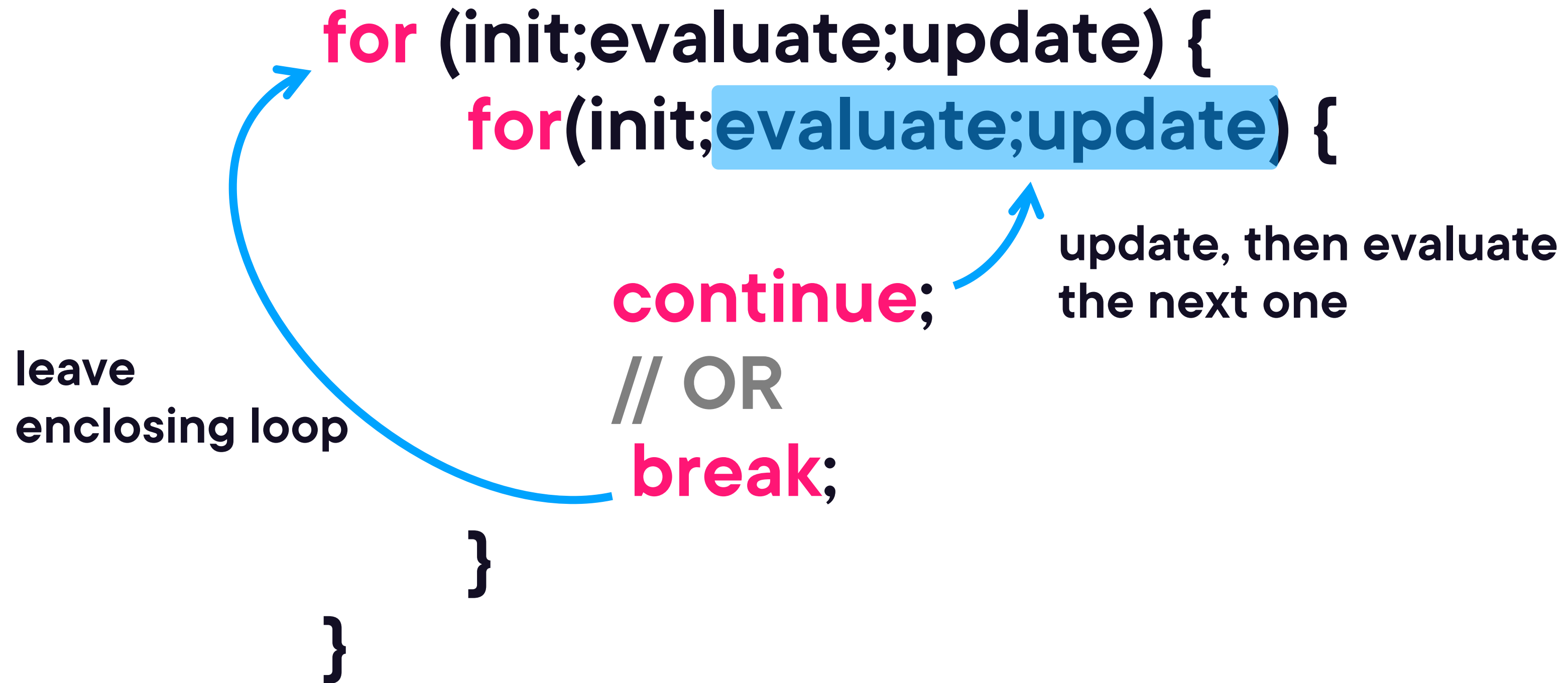
```
int[][] nestedArray = { {1, 2}, {3, 4}, {5, 6} };

OUTER: for (int i = 0; i < nestedArray.length; i++) {
    INNER: for (int j = 0; j < nestedArray[i].length; j++) {
        if (nestedArray[i][j] == 3){
            continue;
        }
        System.out.print(nestedArray[i][j] + " ");
    }
    System.out.println();
}
```

Output

1 2
4
5 6







return = break

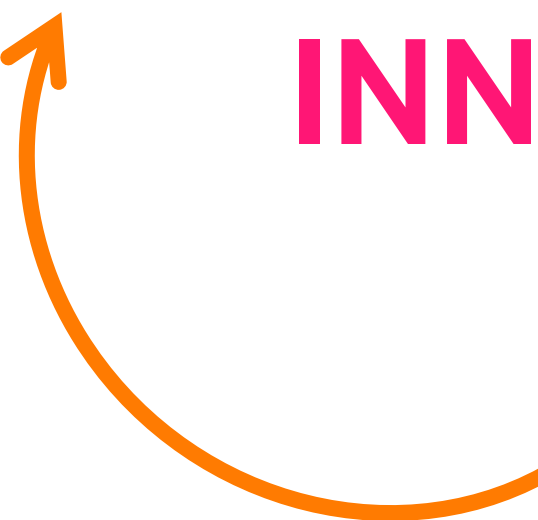
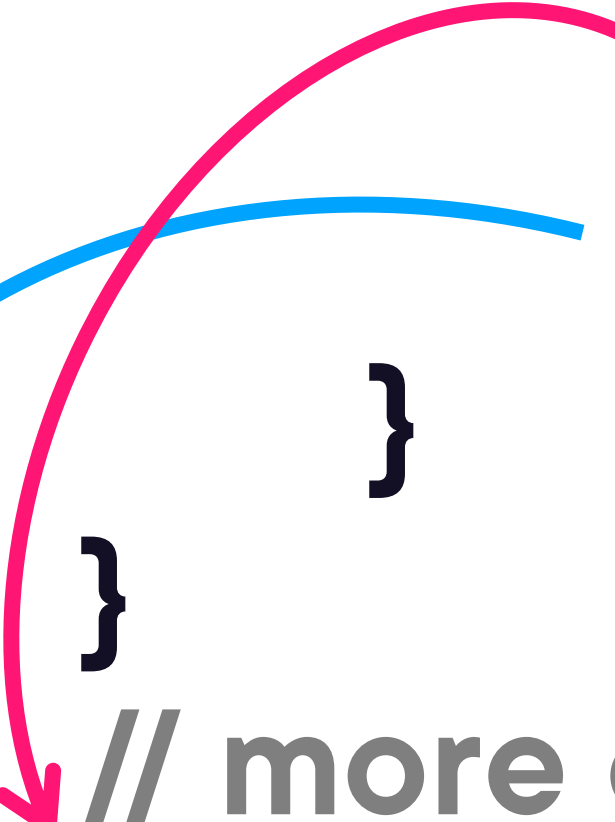
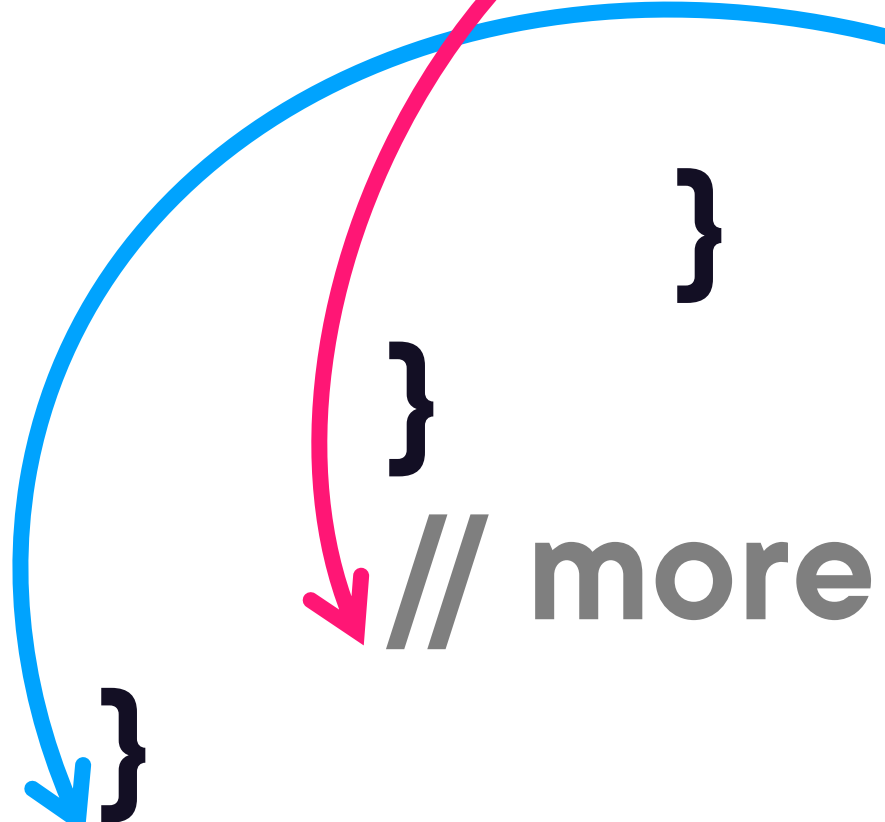


```
void doThing() {  
    for (init;evaluate;update) {  
        for(init;evaluate;update) {  
            return;  
            // OR  
            break;  
        }  
    }  
}
```

leave
enclosing loop

leave method

The diagram illustrates control flow in a nested loop structure. An orange arrow originates from the 'break;' statement and points to the 'for (init;evaluate;update) {' line, indicating it exits the inner loop. A pink arrow originates from the 'return;' statement and points to the closing brace of the method, indicating it exits the entire method. A second pink arrow originates from the closing brace of the inner loop and points to the closing brace of the outer loop, indicating it exits the enclosing loop.

```
void doThing() {  
    OUTER: for (init;evaluate;update) {  
        INNER: for(init;evaluate;update) {  
            continue;   
            break;   
            break OUTER;   
            return;   
        }  
    }  
    // more code  
}
```



Rating



Thank you! Happy coding

